”Humbug” Music-Store

Tienda de Musica en Linea

Materia:	Desarrollo Web
Profesor:	Dr. en C. Margarita Mondragon Arellano
Carrera:	Licenciatura en Informatica y Tecnologias Computacionales
Semestre:	6o.
Nombre del proyecto:	Proyec_Tienda_musica — ASP.NET Core Razor Pages
Alumno:	Liev Fabricio Torres Trinidad

Contenido

1. Crear el proyecto en Visual Studio
2. Crear carpeta Modelos y clase Tienda.cs
3. Crear DBContext.cs
4. Instalar paquetes NuGet
5. Configurar appsettings.json
6. Configurar Program.cs (Inyeccion de Dependencias)
7. Ejecutar migraciones (Add-Migration / Update-Database)
8. Crear las paginas Razor (infotienda, Menu, Recomendaciones, ABCinstrumentos)
9. Actualizar _Layout.cshtml (Navbar)
10. Codigo del PageModel ABCinstrumentos.cshtml.cs (CRUD)
11. Codigo de ABCinstrumentos.cshtml (Vista CRUD)
12. Diseño: estilo The Strokes
13. Extra — API REST con Swagger
14. Extra — Pagina Catalogo.cshtml (Frontend que consume la API)

1 Crear el proyecto en Visual Studio

1. Abrir Visual Studio.
2. Ir a Archivo > Nuevo > Proyecto.
3. Seleccionar la plantilla: Aplicacion web de ASP.NET Core.
4. En la siguiente pantalla colocar los siguientes datos:

Campo	Valor
Nombre del proyecto	*Proyec_Tienda_musica*
Framework	.NET 8.0 (Compatibilidad a largo plazo)
Autenticacion	Ninguno
Configurar para HTTPS	Activado

5. Dar clic en Crear. Visual Studio genera la estructura base del proyecto con las carpetas Pages, wwwroot y los archivos Program.cs y appsettings.json.

2 Crear carpeta Modelos y clase Tienda.cs

1. En el Explorador de soluciones, dar clic derecho sobre el proyecto > Agregar > Nueva carpeta. Nombrarla Modelos.
2. Clic derecho sobre la carpeta Modelos > Agregar > Nuevo elemento > Clase. Nombre: Tienda.cs.
3. Escribir el siguiente codigo:

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace Proyec_Tienda_musica.Modelos
{
    public class Tienda
    {
        [Key]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        public int Id { get; set; }

        public string? Instrumento_Nombre { get; set; }
        public string? Marca { get; set; }
        public string? Descripcion { get; set; }
        public decimal? Precio { get; set; }
        public int? Existencia { get; set; }
    }
}
```

Explicacion de los atributos:

[Key]: indica que Id es la clave primaria de la tabla en SQL Server.

[DatabaseGenerated(DatabaseGeneratedOption.Identity)]: el valor de Id se genera automaticamente (campo autoincremental).

El signo ? en los tipos indica que el campo acepta valores nulos.

3 Crear DBContext.cs

Dentro de la carpeta Modelos, agregar una nueva clase llamada DBContext.cs con el siguiente codigo:

```
using Microsoft.EntityFrameworkCore;

namespace Proyec_Tienda_musica.Modelos
{
    public class DBContext : DbContext
    {
        public DbSet<Tienda> Instrumentos { get; set; }

        public DBContext(DbContextOptions<DBContext> options) : base(options) { }

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            modelBuilder.Entity<Tienda>()
                .Property(t => t.Precio)
                .HasColumnType("decimal(18,2)");
        }
    }
}
```

Explicacion:

DBContext hereda de DbContext, la clase base de Entity Framework Core.

DbSet<Tienda> representa la tabla Instrumentos en la base de datos SQL Server.

El constructor recibe DbContextOptions para que la inyeccion de dependencias funcione correctamente.

OnModelCreating configura la precision decimal del campo Precio para evitar advertencias al migrar.

4 Instalar paquetes NuGet

1. Clic derecho en el proyecto > Administrar paquetes NuGet > Examinar.
2. Buscar e instalar los siguientes paquetes:

Paquete	Proposito
Microsoft.EntityFrameworkCore.SqlServer	Proveedor de SQL Server para EF Core
Microsoft.EntityFrameworkCore.Tools	Habilita comandos Add-Migration y Update-Database
Swashbuckle.AspNetCore	Genera la interfaz Swagger para la API REST

5 Configurar appsettings.json

Abrir appsettings.json y agregar la cadena de conexion a SQL Server Express:

```
{
  "ConnectionStrings": {
    "TiendaDB": "Server=.\\SQLEXPRESS;Database=TiendaMusicaDB;
                Trusted_Connection=True;TrustServerCertificate=True;
                MultipleActiveResultSets=true"
  },
  "Logging": {
    "LogLevel": { "Default": "Information", "Microsoft.AspNetCore": "Warning" }
  },
  "AllowedHosts": "*"
}
```

Nota importante: TrustServerCertificate=True es necesario en versiones recientes de SQL Server para evitar el error de certificado SSL durante la conexion.

6 Configurar Program.cs — Inyeccion de Dependencias

Reemplazar el contenido de Program.cs con el siguiente codigo:

```
using Proyec_Tienda_musica.Modelos;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Servicios de Razor Pages
builder.Services.AddRazorPages();

// Inyeccion de Dependencias — registra DBContext con SQL Server
builder.Services.AddDbContext<DBContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("TiendaDB")));

// Servicios para la API REST y Swagger
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error");
```

```
        app.UseHsts();
    }

    app.UseSwagger();
    app.UseSwaggerUI();
    app.UseHttpsRedirection();
    app.UseStaticFiles();
    app.UseRouting();
    app.UseAuthorization();
    app.MapRazorPages();
    app.MapControllers();
    app.Run();
```

Explicacion:

AddDbContext<DBContext>: registra el contexto en el contenedor de IoC de ASP.NET Core. Esto permite inyectarlo en los PageModels mediante el constructor.

AddControllers y AddSwaggerGen: habilitan la API REST y la interfaz visual de Swagger.

MapControllers: enruta las peticiones HTTP hacia los controladores de la API.

7 Ejecutar migraciones

Las migraciones crean la tabla Instrumentos en SQL Server a partir de la clase Tienda.

1. Abrir la Consola del Administrador de paquetes: Herramientas > Administrador de paquetes NuGet > Consola del Administrador de paquetes.
2. Ejecutar en orden:

```
Add-Migration InitialCreate
Update-Database
```

Si el campo decimal generaba advertencias, se ejecuto una segunda migracion:

```
Add-Migration FixPrecision
Update-Database
```

Resultado esperado: el mensaje 'Build succeeded' y sin errores en rojo confirman que la base de datos TiendaMusicaDB fue creada correctamente en SQL Server.

8 Crear las paginas Razor

Clic derecho sobre la carpeta Pages > Agregar > Nuevo elemento con scaffolding > Pagina de Razor vacia. Crear las siguientes paginas:

Pagina	Proposito
infotienda.cshtml	Informacion de la tienda: historia, productos y contacto
Menu.cshtml	Catalogo de productos por categoria con precios
Recomendaciones.cshtml	Guia de instrumentos por nivel: principiante, intermedio, avanzado
ABCinstrumentos.cshtml	Pagina CRUD: Altas, Consultas, Actualizaciones y Bajas con EF + SQL Server
Catalogo.cshtml	Frontend que consume la API REST para mostrar los instrumentos

9 Actualizar _Layout.cshtml — Navbar

Abrir Pages/Shared/_Layout.cshtml y localizar el bloque del navbar. Reemplazarlo con:

```
<nav class="navbar-strokes">
    <a class="navbar-brand-strokes" href="/">MusicStore</a>
    <ul class="nav-links">
        <li><a asp-page="/Index">Home</a></li>
        <li><a asp-page="/infotienda">Info</a></li>
        <li><a asp-page="/Menu">Menu</a></li>
        <li><a asp-page="/Recomendaciones">Recomendaciones</a></li>
        <li><a asp-page="/ABCinstrumentos">Instrumentos</a></li>
        <li><a asp-page="/Catalogo">Catalogo</a></li>
    </ul>
</nav>
```

10 ABCinstrumentos.cshtml.cs — PageModel CRUD

Este archivo contiene la logica del CRUD. Maneja tres handlers POST: Agregar, Editar y Eliminar.

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Proyec_Tienda_musica.Modelos;

namespace Proyec_Tienda_musica.Pages
{
    public class ABCinstrumentosModel : PageModel
    {
        private readonly DBContext _context;
        public List<Tienda> Instrumentos { get; set; } = new List<Tienda>();

        [BindProperty]
        public Tienda NuevoInstrumento { get; set; }

        [BindProperty]
        public Tienda InstrumentoEditar { get; set; }
```

```
        public ABCinstrumentosModel(DBContext context)
        {
            _context = context;
        }

        // GET: carga todos los instrumentos de la BD
        public void OnGet()
        {
            Instrumentos = _context.Instrumentos.ToList();
        }

        // POST Alta
        public IActionResult OnPostAgregar()
        {
            _context.Instrumentos.Add(NuevoInstrumento);
            _context.SaveChanges();
            return RedirectToPage();
        }

        // POST Actualizacion
        public IActionResult OnPostEditar()
        {
            var item = _context.Instrumentos.Find(InstrumentoEditar.Id);
            if (item != null)
            {
                item.Instrumento_Nombre = InstrumentoEditar.Instrumento_Nombre;
                item.Marca              = InstrumentoEditar.Marca;
                item.Descripcion        = InstrumentoEditar.Descripcion;
                item.Precio             = InstrumentoEditar.Precio;
                item.Existencia         = InstrumentoEditar.Existencia;
                _context.SaveChanges();
            }
            return RedirectToPage();
        }

        // POST Baja
        public IActionResult OnPostEliminar(int id)
        {
            var item = _context.Instrumentos.Find(id);
            if (item != null)
            {
                _context.Instrumentos.Remove(item);
                _context.SaveChanges();
            }
            return RedirectToPage();
        }
    }
}
```

Puntos clave:

[BindProperty]: enlaza automaticamente los datos del formulario HTML con los objetos NuevoInstrumento e InstrumentoEditar.

OnGet recupera todos los registros con ToList() y los asigna a la lista Instrumentos.

Cada handler POST termina con RedirectToPage() para recargar la pagina y mostrar los cambios.

11 ABCinstrumentos.cshtml — Vista CRUD

La vista tiene dos secciones: formulario de alta y tabla de inventario. La edicion se muestra/oculta con JavaScript al presionar el boton Editar de cada fila.

Estructura de la pagina:

Elemento	Descripcion
form asp-page-handler='Agregar'	Formulario con 5 campos para insertar un nuevo instrumento
table.strokes-table	Tabla con todos los instrumentos traidos de la BD
form asp-page-handler='Eliminar'	Boton por fila que envia DELETE con el id como route param
form asp-page-handler='Editar'	Fila oculta con campos prellenados para actualizar el registro
toggleEditar(id)	Funcion JavaScript que muestra u oculta la fila de edicion

12 Diseno: estilo The Strokes

El diseno se basa en la estetica del sitio oficial de The Strokes: fondo azul royal intenso, tipografia pixelada y contrastes en amarillo y blanco.

Elemento	Valor CSS	Descripcion
Fondo	*background-color: #1a1aee*	Azul royal intenso en toda la pagina
Titulo principal	*font-family: 'Press Start 2P'*	Tipografia pixelada de Google Fonts
Texto cuerpo	*font-family: 'VT323'*	Tipografia monoespacio estilo retro
Acento	*color: #ffdd00*	Amarillo para subtitulos y bordes destacados
Tarjetas	*border: 2px solid #fff*	Borde blanco cuadrado sin redondear
Botones	*background: #fff; color: #1a1aee*	Blanco con texto azul, hover en amarillo

Las fuentes se cargan desde Google Fonts en el <head> del _Layout.cshtml:

```
<link href="https://fonts.googleapis.com/css2?family=Press+Start+2P&family=VT323&display=swap"
rel="stylesheet">
```

13 Extra — API REST con Swagger

1. Crear la carpeta Controllers en la raiz del proyecto.
2. Agregar un nuevo elemento: Controlador de API en blanco, nombre InstrumentosController.cs.
3. Escribir el siguiente codigo:

```
using Microsoft.AspNetCore.Mvc;
using Proyec_Tienda_musica.Modelos;

namespace Proyec_Tienda_musica.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class InstrumentosController : ControllerBase
    {
        private readonly DBContext _context;

        public InstrumentosController(DBContext context)
        {
            _context = context;
        }

        // GET api/Instrumentos — devuelve todos los registros
        [HttpGet]
        public IEnumerable<Tienda> Get() => _context.Instrumentos.ToList();
    }
}
```

Para verificar que funciona, ejecutar la aplicacion y navegar a: https://localhost:{puerto}/swagger

Swagger muestra el endpoint GET /api/Instrumentos con un boton 'Try it out' para probarlo directamente desde el navegador.

14 Extra — Catalogo.cshtml: Frontend que consume la API

Esta pagina Razor no usa el PageModel para traer datos de la BD. En su lugar, usa JavaScript (fetch) para llamar a la API REST y construir las tarjetas de forma dinamica.

Codigo de Catalogo.cshtml:

```
@page
@model Proyec_Tienda_musica.Pages.CatalogoModel
@{ ViewData["Title"] = "Catalogo"; }

<div class="page-header">
    <h1>Catalogo</h1>
    <p class="subtitle">— Consulta en tiempo real —</p>
</div>

<div class="content-wrapper">
    <div id="contenedor" class="row g-4">
        <p>Cargando instrumentos...</p>
    </div>
```

```
</div>

@section Scripts {
<script>
    fetch('/api/Instrumentos')
        .then(r => r.json())
        .then(data => {
            const cont = document.getElementById('contenedor');
            cont.innerHTML = '';
            data.forEach(item => {
                cont.innerHTML += `
                    <div class="col-md-4">
                        <div class="strokes-card">
                            <h3>${item.instrumento_Nombre}</h3>
                            <p><strong>Marca:</strong> ${item.marca}</p>
                            <p><strong>Descripcion:</strong> ${item.descripcion}</p>
                            <p><strong>Precio:</strong> $${item.precio} MXN</p>
                            <p><strong>Existencia:</strong> ${item.existencia} uds.</p>
                        </div>
                    </div>`;
            });
        });
</script>
}
```

Explicacion del flujo:

1. Al cargar la pagina, fetch('/api/Instrumentos') hace una peticion GET a la API REST.
2. La API consulta la base de datos y devuelve un JSON con todos los instrumentos.
3. JavaScript recorre el array JSON y genera una tarjeta HTML por cada instrumento.
4. Las tarjetas se insertan dinamicamente en el div#contenedor.

Resumen de archivos del proyecto

Archivo	Descripcion
Modelos/Tienda.cs	Modelo/entidad con 5 campos y data annotations
Modelos/DBContext.cs	DbContext con DbSet<Tienda> y configuracion decimal
appsettings.json	Cadena de conexion a SQL Server con TrustServerCertificate
Program.cs	DI, Swagger, Razor Pages y MapControllers
Pages/_Layout.cshtml	Plantilla maestra con navbar y CSS estilo The Strokes
Pages/Index.cshtml	Pagina de inicio con tarjetas de categorias
Pages/infotienda.cshtml	Historia, productos y contacto de la tienda
Pages/Menu.cshtml	Catalogo de precios por categoria
Pages/Recomendaciones.cshtml	Guia de instrumentos por nivel

Pages/ABCinstrumentos.cshtml	Vista del CRUD completo
Pages/ABCinstrumentos.cshtml.cs	PageModel con OnGet, OnPostAgregar, OnPostEditar, OnPostEliminar
Pages/Catalogo.cshtml	Frontend que consume la API con fetch()
Controllers/InstrumentosController.cs	API REST con endpoint GET /api/Instrumentos
Migrations/	Carpeta generada por EF Core con las migraciones